Ders 7 Genişletilmiş Notu: Metin Ön İşleme, Precision-Recall, Ters İndeks ve Web Crawler'lara Giriş

Ders 7 Genişletilmiş Notu: Metin Ön İşleme, Precision-Recall, Ters İndeks ve Web Crawler’lara Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yeri ve Bilgi Erişimi Sisteminin Tekrarı
- 2. Metin Ön İşleme
- 3. TF-IDF ve Doküman-Terim Matrisi
- 4. Bilgi Erişimi Başarısının Ölçülmesi
- 5. Precision-Recall Eğrisi
- 6. Ters İndeksin Yapısı
- 7. Ters İndeks Üzerinde Arama
- 8. Sorgu Genişletme
- 9. Bölümde Ayrıntısına Girilmeyen Konular
- 10. Web Crawling: Motivasyon ve Temel Fikir
- 11. Gerçek Crawler’lar, Bant Genişliği ve robots.txt
- 12. Crawler’ların Kullanım Alanları
- 13. Crawler ve Arama Motoru Mimarisi
- 14. Crawler Sınıflandırması
- 15. Crawler ve Scraper Sorusu
- 16. Proje Önerisi Görüşmesi

1. Dersin Yeri ve Bilgi Erişimi Sisteminin Tekrarı

Hoca derse, kitapta nerede olduğumuzu söyleyerek başladı. Kısmi gözetimli öğrenme (partially supervised learning) algoritmalarının anlatıldığı 5. bölüm dışındaki bölümler sırayla işlenmişti. Bu derste bilgi erişimi ve web araması (information retrieval and web search) bölümünün konuşulmayan kısımları bitirilecek, kalan zamanda web crawling bölümüne girilecekti.

Önceki haftada iki soru konuşulmuştu: Bilgi erişiminin veritabanı erişiminden (database retrieval) farkı nedir? Bir bilgi erişimi uygulamasında hangi modüller vardır ve bunlar birbiriyle nasıl etkileşir? Hoca yeni konuya geçmeden önce bu mimariyi baştan sona bir kez daha izledi.

1.1 Doküman Koleksiyonu

Bir bilgi erişimi sistemini besleyen uygulama crawler’dır. Crawler dağıtık ortamdaki dosyaları alır ve bir doküman koleksiyonu (document collection) oluşturur. Bu işin nasıl yapıldığı sonraki bölümün konusudur.

Her bilgi erişimi motoru dağıtık ortamdaki dokümanlar üzerinde çalışmaz. Bazen yalnız kendi çalıştığınız yerel ortamdaki dokümanlarda arama yapmak istersiniz. İki durumda da elimizde merkezi bir ortamda duran bir doküman koleksiyonu olduğunu düşünürüz.

1.2 Doküman İndeksi

Arama, doküman koleksiyonunun kendisi üzerinde yapılmaz. Koleksiyonun indekslenmesi gerekir. İndeks iki kolonlu bir tablo gibi düşünülebilir:

- Birinci kolonda aramanın yapılacağı anahtar kelimeler (keyword) vardır.
- İkinci kolonda her anahtar kelimeyi içeren dokümanların sıralanmış listesi vardır.

Sıralama, kelimenin o doküman içindeki önemini gösteren TF-IDF skoruyla yapılır. Yani bir kelimeyi içeren bütün dokümanlar TF-IDF skorlarına göre sıraya dizilir. Hoca bu sayede TF-IDF'in tam olarak nerede kullanıldığını da hatırlatmış oldu: İndeksleyici (indexer), doküman indeksini oluştururken her anahtar kelimenin dokümanlarını sıralamak için bu ağırlığı kullanır.

Hoca bu tabloya biraz sonra ters indeks (inverted index) diyeceğimizi söyledi. Adın sebebi şudur: Bu tablo kelimelerden dokümanlara ulaşmamızı sağlar.

1.3 Sorgudan Sonuca

Kullanıcı bir anahtar kelime ya da anahtar kelime grubu verir. Sistem bunu çalıştırılabilir sorguya (executable query) dönüştürür. Farklı tipteki kullanıcı sorgularının nasıl dönüştürüldüğü önceki haftanın konusuydu.

Hoca önce çalıştırılabilir sorgu ile indeks arasında bir benzerlik fonksiyonu çalıştığını söyledi, sonra kendi ifadesini daha doğru hale getirdi: Sorgudaki kelimeler için tabloda arama (lookup) yapılır. Tabloyu bir karma tablosu (hash table) olarak tuttuğumuzu düşünebiliriz. Girilen kelimelerin bulunduğu girdiler (entry) bulunur. Bu girdilerdeki doküman zincirleri alınır, birleştirilir ve sıralanmış olarak son kullanıcıya gönderilir.

Hoca slayttaki mimaride, geçen haftadan beri konuşulan kısmın arama motorunun doküman indeksi ile sorgu işlemleri (query operations) tarafı olduğunu belirtti. Akış düz metin olarak şöyle özetlenebilir:

```
crawler -> doküman koleksiyonu -> indeksleyici -> ters indeks
                                                      ^
kullanıcı sorgusu -> çalıştırılabilir sorgu -> erişim motoru -> sıralı sonuçlar
```

1.4 Erişim Modelleri ve Benzerlik

Bir bilgi erişimi modeli, sorgu ile dokümanı eşleştirir. Sorgunun da dokümanın da bir iç temsili (internal representation) vardır. Bu iki temsil arasındaki benzerlik, modelin belirlediği fonksiyonla ölçülür.

- **Vektör uzayı modeli (vector space model):** Benzerlik kosinüs benzerliği (cosine similarity) ile ölçülür.
- **Boolean bilgi erişimi:** Kelime ya da kelime grubunun dokümanda kesin olarak bulunması aranır. AND ve OR kullanılır. Bir kelimenin geçmemesi istenirse NOT operatörü kullanılır. Girilen ifadeyi tam eşleşmeyle (exact match) karşılayan dokümanlar ters indeks tablosunda yerini alır.

  Ders sonrası not: Hoca, tablodaki dokümanlar arasında benzerlik bulunarak dokümanların ilgili anahtar kelimenin girdisine konduğunu ve tablo oluşturulurken kosinüs benzerliği kullanıldığını söyledi. Doğrusu büyük olasılıkla şudur: Ters indeks, kelimenin dokümanda geçmesine ve TF-IDF ağırlığına göre kurulur. Kosinüs benzerliği ise sorgu anında sorgu vektörü ile doküman vektörü arasında hesaplanır.

2. Metin Ön İşleme

Dokümanlar indekslenmeden önce üzerlerinde bir ön işleme (text preprocessing) yapılır. Hoca nedenini somut bir durumla açıkladı: İnternetten bir HTML dokümanını olduğu gibi indirdiğinizi düşünün. Bu dokümanın içinde URL'ler ve resimler vardır. Resmin kendisi HTML'in içine gömülü değildir. HTML indirilirken ayrı bir dosya olarak iner. HTML'in içinden resme bir bağlantı (link) gider.

Önceki derste bir dokümanın, içinden çıkarılan kelimelerle kelime torbası (bag of words) biçiminde, yani bir vektör olarak ifade edilebileceği söylenmişti. Hem Boolean model hem vektör uzayı modeli dokümanları artık birer vektör olarak ele alır. Kelimeler çıkarılmadan önce doküman üzerinde yapılması gereken işlemler bu bölümün konusudur.

2.1 Temsilci Vektörün Önemi

Doküman bir vektöre dönüştürüldükten sonra, sorgu ile doküman arasındaki benzerlik ölçülürken artık bu vektör kullanılır. Vektör dokümanın temsilcisidir. Gerçek dokümandan uzaklaşmışızdır ve bütün hesaplar temsilci üzerinden yapılır.

> **Hocanın vurgusu:** Dokümanı temsil edecek vektör gerçekten doğru oluşturulmalıdır. Aksi halde dokümanın eşleştiği sorguları bulmakta yanlış yaparız, çünkü artık bütün hesaplamalar gerçek dokümanla değil temsilcisiyle yapılmaktadır.

Vektör çıkarılırken yapılan işlemler sırasıyla şunlardır:

- Terimleri çıkarma
- Durak kelimeleri çıkarma
- Kök bulma
- TF-IDF skorlarını hesaplama

2.2 Terim Çıkarma

İlk adım terimleri bulup çıkarmaktır (extraction). Bir ayrıştırıcı (parser) yazarak verilen bir PDF, Word ya da HTML dokümanından kelimeleri çıkarmak kolaydır. Hocaya göre burada bir sıkıntı yoktur.

2.3 Durak Kelimeleri Çıkarma

Çıkarılan kelimelerin arasında durak kelimeler (stop words) de vardır. Bunlar kendi başına anlamı olmayan, ama dilin gramerinde görevi olan, başka kelimelerle birlikte kullanılarak cümle kurmaya yarayan kelimelerdir. Hocanın örnekleri şunlardı: Türkçede bağlaçlar, İngilizcede “the” artikeli, Almancada “der, die, das” artikelleri. Her dilde böyle kelimeler vardır. Ancak birlikte kullanıldıkları kelimelerle bir anlam oluştururlar.

Dokümanı vektöre dönüştürürken bu kelimelerin bir faydası yoktur. Vektöre yalnız kendi başına anlamı olan ve dokümanı tanımlayan kelimeler alınmak istenir. Bunlar da genelde terimler, yani isimlerdir.

Durak kelimeleri çıkarmanın bir kazancı daha vardır: Oluşturulacak tablonun indeks boyutu küçülür. Hoca, bu işlemin toplam kelime sayısını yaklaşık %20-30 azalttığını ve indeksin de bu yüzden küçüldüğünü söyledi.

2.4 Kök Bulma

Bir sonraki işlem kök bulmadır (stemming). Özellikle sondan eklemeli dillerde aynı kelimenin farklı ek almış biçimleri bulunur. Türkçe böyle bir dildir. Hoca bu biçimlerin her birinin kelime vektöründe ayrı kelime olarak gösterilmesini istemediğimizi söyledi. Aynı anlamı veren bir kelime dokümanda 3-4 farklı ek almış biçimde geçiyorsa, bunların kökü bulunur ve vektörde kök gösterilir.

Hocanın İngilizce örnekleri şunlardı:

- “user, users, used, using”: Bunların hepsi “use” kelimesinden türemiştir.
- “engineering, engineered”: Bunlar da aynı kökten türemiştir.

Dokümanda bu biçimlerle karşılaşılınca kökü bulunur ve dokümanda yalnız bu kökün var olduğu kaydedilir. Kök bulma da indeksi küçültür: Hocaya göre oluşturulacak indeks tablosunun boyutu yaklaşık %40-50 azalır.

Kök bulma kütüphaneleri (stemming libraries) genelde kural tabanlıdır ve her dil için değişir. Sondan ek alan dillerde kelimeye ek geldiğinde kökün son harfleri ekin etkisiyle değişebilir. Bununla ilgili kod yazıyorsanız işi kural tabanlı yapmanız gerekir: Bu türden kaç durum varsa

her birini bir kurala dönüştürürsünüz. Dokümanda böyle bir kelimeyle karşılaşınca ilgili kurala göre kökünü bulursunuz. Slayttaki kurallar yalnız İngilizce içindir.

2.5 Vektörde Frekans Bilgisi

Kelime torbası oluşturulurken yalnız kelimenin dokümanda olup olmadığı değil, kaç kez geçtiği de tutulmalıdır. Hoca bunun için iki gösterim söyledi. Vektörün elemanları birer ikili olarak düşünülebilir: Biri kelimenin olup olmadığını, öbürü kaç kez geçtiğini gösterir. Ya da tek elemanlı bir gösterim kullanılır: O zaman hücrede 1 ya da 0 değil, kelimenin dokümanda kaç kez geçtiğini gösteren sayı olur. Vektör uzayı modelinde durum buydu.

Soru: Doküman web sayfasından mı gelir?

Cevap: Web bağlamında doküman dediğimiz şey zaten web sayfasıdır. Bilgi erişimi bir web arama motoruysa doküman web sayfasıdır. Ama bilgi erişimi kendi makinenizdeki ya da bir kurumun içindeki dokümanlar üzerinde yapılıyorsa, doküman o Word ya da PDF dosyalarıdır.

3. TF-IDF ve Doküman-Terim Matrisi

Stop word çıkarma ve kök bulma bittikten sonra sıra TF-IDF skorlarına gelir. Elimizde toplam doküman seti ve her doküman için kelimeleri gösteren vektörler vardır. Artık skorlar hesaplanabilir. Hesabın nasıl yapıldığı önceki derste görülmüştü. Hoca yalnız anlamını hatırlattı:

- **TF (terim frekansı, term frequency):** Bir kelimenin bir doküman içinde kaç kez geçtiğiyle ilgilidir. Kelimenin o doküman içindeki önemini gösterir. Kelime dokümanda ne kadar sık geçiyorsa TF-IDF skoru o kadar yüksek olur.
- **IDF (ters doküman frekansı, inverse document frequency):** Kelimenin toplam doküman setinde kaç dokümanda geçtiğiyle ilgilidir. Kelimenin bütün doküman setindeki önemini gösterir. Kelime ne kadar az dokümanda geçiyorsa TF-IDF skoru o kadar yüksek olur.

3.1 Vocabulary

Tüm doküman setinden çıkarılan kelimelerin oluşturduğu kelime uzayına vocabulary denir. Burada yalnız bir dokümandaki değil, bütün doküman setindeki benzersiz (unique) kelimeler kastedilir. Vocabulary iki yerde belirleyicidir:

- Vektörün boyutu vocabulary'deki kelime sayısı kadardır.
- Ters indeks tablosunun birinci kolonundaki kelimeler vocabulary'den gelir.

Bir dokümanın vektöründe, kelime o dokümanda varsa ilgili noktaya o kelimenin TF-IDF değeri yazılır.

3.2 Doküman-Terim Matrisi

Her doküman bir vektörle temsil edilince elimizde bütün doküman setini gösteren bir matris olur. Satırlar dokümanlardır. Sütunlar vocabulary'deki anahtar kelimelerdir. Hücrelerde TF-IDF skorları bulunur. Kelime dokümanda varsa hücrede sıfırdan farklı (non-zero) bir değer olur.

	kelime_1	kelime_2	...	kelime_V
doküman_1	tfidf	0	...	tfidf
doküman_2	0	tfidf	...	0
...				
doküman_N	tfidf	0	...	tfidf

> **Ders sonrası not:** Hoca, kelime dokümanda varsa hücrede her zaman sıfırdan farklı bir değer olacağını söyledi. Bu büyük olasılıkla genel durum için söylenmiştir. IDF, $\log(N/df)$ biçiminde tanımlanırsa, bütün dokümanlarda geçen bir kelimede $\log(N/N) = 0$ olur. Bu durumda kelime dokümanda bulunsa da TF-IDF değeri 0 çıkar.

3.3 Matristen Ters İndekse

İndeksleyici bu matristen yola çıkarak tabloyu oluşturur. Tablonun birinci kolonu kelimelerdir. İkinci kolonu ise o kelimeyi içeren dokümanların TF-IDF skoruna göre, en büyükten en küçüğe sıralanmış bağlı listesidir (linked list).

Hoca burada bir noktaya dikkat çekti: Doküman listesi derken listedeki düğümler (node) dokümanın kendisi değildir. Düğüm, dokümanı işaret eden, dokümanın ID'sini içeren bir özet bilgidir.

Erişim sistemi sorguyu aldıktan sonra bu tabloya gider, ilgili kelimeyi içeren dokümanları alır, sıralanmış bir doküman listesi oluşturur ve ilk 10'unu son kullanıcıya gösterir.

4. Bilgi Erişimi Başarısının Ölçülmesi

Sorguları alıp sonuçları doküman indeksinden getiren bir erişim motorumuz (retrieval engine) var. Bu motorun işini ne kadar başarılı yaptığı ölçülmelidir.

4.1 Başlangıç Örneği

Hocanın örneği şöyleydi: Doküman setinde 100 doküman var ve "web mining" aranıyor. Motor, içinde web mining geçen dokümanları getirecek, ama yalnız 10 tanesini getirmesi isteniyor. Getirilen 10 dokümandan kaçının gerçekten web mining ile ilgili (relevant) olduğuna karar verilir. Bu örnekte ilgili doküman sayısı verilmemiştir. Örnek yalnız ölçümün mantığını kurmak içindir.

> **Ders sonrası not:** Hoca, önce getirilen dokümanlar içinde "relevant olanların sayısına" precision dedi, birkaç dakika sonra doğru tanımı A ve B kümeleriyle oran olarak verdi. Doğrusu orandır: Precision, getirilen dokümanlar içinde ilgili olanların oranıdır.
>
> **Ders sonrası not:** Hoca ilgili dokümanları bazen "web mining'i içeren dokümanlar" diye anlattı. Altyazıda aynı örnekte getirilen dokümanların "web mining ile ilgili olup olmadığına karar verileceği" de söyleniyor. Bu yüzden kastedilen büyük olasılıkla ilgililiktir. İlgili olmak ile kelimeyi içermek aynı şey değildir.

4.2 Precision ve Recall'un Cevapladığı İki Soru

Bir sorgu verildiğinde bu iki metrik iki soruya cevap verir:

- **Kesinlik (precision):** "Are all retrieved documents relevant?" Yani getirdiğin dokümanların hepsi ilgili mi, yüzde kaçı ilgili?
- **Duyarlılık (recall):** "Have all the relevant documents been retrieved?" Yani doküman setinde bu sorgu için ilgili olan bütün dokümanları düşünürsek, hepsini getirebildin mi?

4.3 Küme Gösterimi

Hoca slayttaki şekli şöyle anlattı: Mavi kare toplam doküman koleksiyonudur. İçinde iki "yumurta" vardır.

- Mor yumurta, verilen sorgunun karşılığı olan dokümanlardır. Bu kümeye A diyelim.
- Yeşil yumurta, erişim motorunun sorguya karşılık getirdiği dokümanlardır. Bu kümeye B diyelim.

Kullanıcıya gösterilen dokümanlar yeşil yumurtanın içindekilerdir. Bunların kaçının sorguyla ilgili olduğu iki kümenin kesişimiyle bulunur. Formüller şöyledir:

$$precision = \frac{|A \cap B|}{|B|} \qquad recall = \frac{|A \cap B|}{|A|}$$

Precision, getirilen dokümanların yüzde kaçının ilgili olduğunu söyler. Recall, motorun getirdiği ilgili dokümanların, toplam ilgili dokümanların içinde yüzde kaç olduğunu söyler. Bir erişim motorunun başarısı bu ikisi kullanılarak ölçülür.

4.4 Precision@k

Precision'ın farklı kaynaklarda ve kitapta farklı değerleri vardır. Precision Top 10, Top 20, Top 30 kullanılabilir. Bu, son kullanıcıya gösterilen ilk 10, ilk 20 ya da ilk 30 sonuç içindeki precision demektir. Hoca top 5, 10, 15, 20, 25 gibi değerleri de saydı. Yani precision tek başına kullanılmaz, bir sonuç sayısıyla birlikte verilir. Bu notta Precision Top 10 için Precision@10 yazılır.

4.5 Web Ortamında Ölçüm

Motor web ortamında çalışıyorsa çözülmesi gereken problem sayısı artar. Burada motora bir iki anahtar kelime verilir ve çok büyük ölçekli bir veri setinde en doğru cevapları bulması istenir. Üstelik aranan dokümanların ilk 10 sonuçta çıkması beklenir.

> **Hocanın vurgusu:** Web ortamında başarı genellikle Precision@10 ile ölçülür. Bir arama yaptığınızda genelde ilk 10 sonuca bakarsınız. Aradığınızı bulamazsanız anahtar kelimenizi değiştirirsiniz. İlk 20, 30, 40 sonuca bakan kullanıcı sayısı azdır.
>
> **Hocanın vurgusu:** Recall web ortamında ölçülemez. Bir kelimenin web'de gerçekte kaç karşılığı olduğunu bilemeyiz.

Hoca bunu formül üzerinden gösterdi. Pay bilinebilir: İlk 10 sonuçtan kaçının ilgili olduğu tespit edilebilir. Payda ise bilinemez: Bir sorguyla gerçekten ilgili olan bütün dokümanları web ölçeğinde bulmak imkansızdır. Bu yüzden web ortamında hesaplanabilen precision kullanılır.

Motor yerelde, kendi doküman setiniz üzerinde çalışıyorsa durum değişir. Aranan kelimeye karşılık kaç ilgili doküman olduğu tespit edilebilir. Buradan recall da hesaplanabilir.

4.6 Çeşitlilik

Hoca bir başarı ölçüsü daha söyledi: çeşitlilik (diversity). Arama motorları karşılaştırılırken yalnız ilgili dokümanların ilk 10'da gelip gelmediğine bakılmaz. Sonuçların farklı alan adlarından (domain) gelip gelmediğine de bakılır.

Hocanın örneği şöyleydi: Bing'de bir arama yaptığınızı, sonuçların hepsinin aynı alan adından geldiğini düşünün. Google'da aynı aramayı yaptığınızda ise sonuçlar farklı alan adlarından geliyor. Bu durumda ikinci motorun sonuçlarına daha çok güvenirsiniz. Çünkü size tek bir sayfanın görüşünü empoze etmeye çalışmıyor. Kelimenize karşılık gelen bilgileri farklı sayfalardan gösteriyor. Hoca bu örneği iki motoru ölçmek için değil, ölçütü anlatmak için varsayımsal olarak kurdu.

5. Precision-Recall Eğrisi

Bir arama motorunun başarısı genelde precision-recall grafiğiyle gösterilir. Grafikte X ekseni recall, Y ekseni precision'dır. X eksenindeki recall değerleri sıfırdan başlatılır ve %100'e kadar %10'ar artırılır. Her recall seviyesinde precision'ın kaç olduğu bulunur.

5.1 Hocanın Örneği

Verilen: Doküman setinde 100 doküman vardır. Girilen sorguyla ilgili doküman sayısı 10'dur. Yani $|A| = 10$. Sonuçlar sırayla gösterilir. Her adımda şu iki değer hesaplanır:

$$recall = \frac{\text{şimdiye kadar gelen ilgili sayısı}}{10} \qquad precision = \frac{\text{şimdiye kadar gelen ilgili sayısı}}{\text{şimdiye kadar gösterilen sonuç sayısı}}$$

Adımlar şöyledir:

- **1. sonuç ilgili:** Recall $1/10 = \%10$, precision $1/1 = \%100$.
- **2. sonuç ilgili:** Recall $2/10 = \%20$. Gösterilen iki sonucun ikisi de ilgili olduğu için precision $2/2 = \%100$.
- **3. sonuç ilgili:** Recall $3/10 = \%30$, precision $3/3 = \%100$.

- **4. sonuç ilgisiz:** Hoca burada precision'ın "yavaş yavaş düşmeye" başladığını söyledi. İlk dört sonucun üçü ilgilidir, bu yüzden precision $3/4 = \%75$ olur (ara adım eklendi). Recall $3/10 = \%30$ olarak kalır (ara adım eklendi).

Bu şekilde recall değerleri artarken karşılık gelen precision değerleri için bir tablo çıkarılır. Grafik bu tablodan çizilir.

> **Ders sonrası not:** Hoca, 4. sonucun ilgisiz olduğunu söyledikten sonra "%40 olduğu zaman precision düşüyor" dedi. Bu iki ifade aynı adım için geçerli değildir. İlgisiz bir sonuç recall'u artırmaz. 4. sonuç ilgisizse o adımda recall %30'da kalır. Recall %40 seviyesine ancak 4. ilgili doküman geldiğinde çıkar, bu da en erken 5. sonuçtur. Dolayısıyla recall %40 seviyesindeki precision en fazla $4/5 = \%80$ olabilir.

5.2 Birden Fazla Sorguyla Ortalama

Bir motorun başarısı tek bir sorguyla ölçülmez. Tek sorgu girip precision-recall değerlerini alarak karşılaştırma yapılmaz. Yapılan iş şudur:

- Çok sayıda sorgu seçilir. Sorguların farklı alanlardan gelmesi tercih edilir.
- Her sorgu için, farklı recall seviyelerindeki precision değerleri hesaplanır. Yani yukarıdaki tablo her sorgu için ayrı çıkarılır.
- Her recall seviyesi için ayrı ayrı ortalama alınır. Örneğin recall'un sıfır olduğu satır için birinci, ikinci, üçüncü tablodaki precision değerlerinin ortalaması alınır ve son tabloya yazılır.
- Precision-recall eğrisi, tek sorgudan gelen tablodan değil, çok sayıda tablonun ortalamasından oluşan bu son tablodan çizilir.

> **Ek bilgi (derste anlatılmadı):** Hocanın sözlü anlattığı ortalama formülle şöyle yazılabilir: $\bar{p}(r_i) = \frac{1}{|Q|} \sum_{j=1}^{|Q|} p_j(r_i)$. Burada Q sorgu kümesi, r_i bir recall seviyesi, $p_j(r_i)$ ise j. sorgunun bu seviyedeki precision değeridir. Slayttaki gösterim bundan farklı olabilir.

Hoca bu noktada nerede olduğumuzu özetledi: Erişim sisteminin çalıştırılabilir sorguyu alıp ters indeks üzerinde sonuçları bulması ve sıralı göstermesi konuşulmuş, bunun başarısının nasıl ölçüleceği de görülmüştü. Sıradaki konu ters indeksin yapısıydı.

6. Ters İndeksin Yapısı

Web ortamında crawler, farklı uygulama sunucuları (hocanın örnekleri IIS ve Tomcat) altında duran HTML dokümanlarını alıp doküman koleksiyonuna koyar. Aradığımızı bu koleksiyonun içinde arama anında bulamayız, çünkü aramanın çok hızlı olması gerekir. Bu yüzden doküman seti, üzerinde çok hızlı arama yapılabilecek bir indekse dönüştürülür.

Arama motorlarının kullandığı bu yapıya ters indeks denir, çünkü dosyalara kelimelerden yola çıkarak ulaşılır. Tablo dosyaları ayırt edici (distinctive) kelimelerle eşleştirir. Bir kelime arandığında lookup yapılır, kelimenin bulunduğu girdi bulunur. Girdideki sıralı doküman listesi alınır ve son kullanıcıya gösterilir.

6.1 Tablonun Kurulması

Hoca slaytta iki tablo gösterdi. Sol tarafta (A) daha basit, sağ tarafta (B) daha ayrıntılı bir ters indeks vardı. İkisi de üç dokümanlık bir set üzerinden kuruluyordu. Adımlar şöyledir:

- Birinci, ikinci ve üçüncü dokümanlardan kelime uzayı çıkarılır. Kelimeler ayırt edici olmalıdır, stop word'ler kullanılmaz.
- Kelime uzayındaki kelimeler tablonun girdileri, yani anahtarları (key) olur.
- Her kelimeyi içeren dokümanlar tablonun değer (value) kısmına yazılır.

6.2 Basit Tablo

Basit tabloda her kelime için yalnız onu içeren dokümanların ID'leri bulunur. Hocanın okuduğu girdiler şunlardır:

```
applications:  id2
hyperlink:     id3
mining:        id1, id2, id3
```

6.3 Ayrıntılı Tablo

Ayrıntılı tablo, kelimenin dokümanda kaç kez geçtiğini ve kaçıncı pozisyonda geçtiğini de verir. Her girdi bir üçlüdür: doküman ID'si, dokümandaki geçme sayısı, pozisyonlar. Hocanın okuduğu girdiler şunlardır:

```
applications:  <id2, 1, [3]>
mining:        <id1, 1, [..]>, <id2, 1, [..]>, <id3, 1, [..]>
structure:     <id3, 2, [2, 8]>
```

- "applications" 2. dokümanda 1 kez geçer. Kelime sıralamasında 3. pozisyondadır.
- "mining" üç dokümanda da 1'er kez geçer. Hoca her birinde pozisyonların farklı olduğunu söyledi, değerleri okumadı.
- "structure" 3. dokümanda 2 kez geçer. Bu yüzden ikinci değer 2'dir. 2. ve 8. sıralarda geçtiği için iki pozisyon yazılır.

6.4 Pozisyonlar ve Ek Bilgi Neden Tutulur?

Hoca iki sebep söyledi. Birincisi yakınlık aramasıdır (proximity search). Önceki derslerde görüldüğü gibi, bir kelime grubu aranırken yakınlık seçeneği işaretlenirse, kelimeleri birbirine yakın olan dokümanlara öncelik verilir. Kelimeler aynı paragrafta ya da aynı cümledeyse o doküman daha önemlidir. Bunu hesaplamak için pozisyon gerekir.

İkincisi, kelimenin doküman içindeki önemini gösteren bilgilerdir. Kelime HTML dokümanında meta etiketi (meta tag) içinde ya da başlıkta olabilir. Kalın (bold) ya da italik yazılmış olabilir. Kelimenin diğerlerinden farklı olduğunu gösteren böyle bir bilgi varsa, pozisyon gibi bu da girdiye yazılır.

Girdi burada bir dokümanı ifade eden zenginleştirilmiş bir gösterimdir. Örnekteki üçlü vektör, eklemek istediğimiz bilgi kadar dörtlü, beşli, altılı olabilir.

7. Ters İndeks Üzerinde Arama

Hoca arama işlemini "web mining" sorgusu üzerinden adım adım izledi.

- **Sözlük araması (vocabulary search):** Sorgu için lookup yapılır ve kelimenin hangi girdide olduğu bulunur. Sorgu birden fazla kelimeliyse her kelime için ayrı gidilir: Önce "web" girdisine, sonra "mining" girdisine.
- **Sonuçları birleştirme (merge):** Slayttaki tabloda "mining" girdisinde 3, "web" girdisinde 2 doküman vardır. Bir kelimeden 3, öbüründen 2 doküman gelir. Gelen sonuçlar TF-IDF skorlarına göre birleştirilir.
- **Yeniden sıralama (re-ranking):** Birleştirilen sonuçlar doğrudan son kullanıcıya gösterilmez. Önce yeniden sıralanır. Web aramasında bu genellikle bağlantı tabanlı (link-based) bir sıralamadır. Bu yöntem bu bölümde değil, başka bir bölümde çalışılacaktır.

  Ders sonrası not: Hoca, birleştirme yapılırken tablodaki girdilerde kelimenin dokümandaki önemini gösteren TF-IDF bilgisinin bulunduğunu söyledi. Ancak okuduğu üçlüler doküman ID'si, geçme sayısı ve pozisyondan oluşuyor; TF-IDF bu üçlülerde yok. Büyük olasılıkla skor ayrı bir alan olarak tutulur ya da geçme sayısı ve doküman frekansından hesaplanır.

  Hocanın vurgusu: Farklı arama motorlarının farklı başarıda sonuç göstermesinin temel sebebi, ters indeksin içine ne kadar ek bilgi koyduklarıdır. Kelimenin başlıkta mı, gövdede mi, meta etiketinde mi olduğu gibi bilgiler konduğunda, eşleştirme ve sıralama daha başarılı olur.

8. Sorgu Genişletme

Arama motorları, daha başarılı sonuç almak için kullanıcıdan aldıkları sorguyu genişletir (query expansion). Yani sorgudaki kelimelerin yanına, kullanıcının önceki aramalarından yola çıkarak ek kelimeler koyar.

Hocanın örneği şöyleydi: "web mining" yazmadan önce "İstanbul", "Yıldız Teknik Üniversitesi" ve İstanbul'daki lokantaları aradığınızı düşünün. Bütün bunlardan sonra "web mining" girdiğinizde akıllı bir arama motoru, Çin'deki bir üniversitede verilen web mining dersini ilk 10 sonuçta getirmez. Bulunduğunuz konumu, girdiğiniz kelimelerden hangi şehirde ve ülkede olduğunuzu izler. Web mining geçen sonuçları bu bilgiye göre getirir.

Motor sorgunuza belki "İstanbul", belki "Türkiye", belki "Yıldız Teknik Üniversitesi" ekler. Ama bu genişletmeyi size göstermez, arka planda yapar. Böylece ilk 10 sonuçta daha çok tercih edeceğiniz sonuçları görürsünüz.

9. Bölümde Ayrıntısına Girilmeyen Konular

Hoca bölümü kapatırken konuşulmayan ya da yüzeysel geçilen konuları saydı:

- **Başka erişim modelleri:** Boolean ve vektör uzayı modelinden başka modeller de vardır. Hoca istatistiksel dil modelini (statistical language model) ve gizli anlamsal indekslemeyi (Latent Semantic Indexing) saydı.
- **İndeks sıkıştırma (index compression):** İndeksi küçük tutmak için stop word çıkarma ve kök bulma konuşuldu. Ama çok büyük ölçekli verilerde ya da çok kelimeli dillerde indeks yine de büyük olur. Bunun için sıkıştırma yöntemleri vardır.
- **Web sayfası ön işleme:** Ayrıntısına girilmedi. Ayrıntılar kitaptan okunabilir.
- **Web spam:** Web spam'in e-posta spam'inden farkı söylendi. Nasıl önleneceğine ayrıntılı girilmedi.
- **Sıralama mekanizmalarının birleştirilmesi:** Farklı sıralama mekanizmalarının nasıl birleştirileceğine girilmedi.

TF-IDF tabanlı sonuçların link analizi sonuçlarıyla birleştirilmesi ise ileride, web yapı madenciliği (web structure mining) konuşulurken ele alınacaktır.

10. Web Crawling: Motivasyon ve Temel Fikir

Aradan sonra hoca web crawling bölümüne geçti. Bilgi erişimi motorlarının yapısı konuşulurken, özellikle web arama motorlarında doküman koleksiyonunu crawler'ın oluşturduğu söylenmişti. Şimdiki soru, bunu nasıl yaptığıydı. Hoca planı şöyle açıkladı: Önce motivasyon, sonra sınıflandırma (taxonomy), yani hangi tipte crawler'lar olduğu. Ardından en temel bir crawler alınıp, onu gerçekleştirmek (implement) isteyince nelere dikkat edileceğine bakılacak.

10.1 Ölçek ve Saklama

Web aramasında çok büyük ölçekli veriyle uğraşılır. Doküman sayısı milyonlar, web aramasında milyarlar seviyesindedir. Bazı arama motorları ise dikey (vertical) arama motorlarıdır: Yalnız elektronik alanındaki sayfalarda ya da yemek tariflerinde arama yaptırırlar. Bunların yönettiği doküman sayısı milyonlar mertebesine iner.

Arama motorları bu dokümanları merkezi bir ortamda toplamak için crawler kullanır. Ancak "merkezi" demek tek düğüm (node) demek değildir. Dokümanlar merkezi ortamda dağıtık sistem mimarisiyle tutulur. Bir veri merkeziniz (data center) vardır. Bu merkezdeki kümede (cluster), dokümanlar çok düğümlü bir NoSQL veri saklama teknolojisiyle saklanır. İndeksleme çalışmaları bunun üzerinde yapılır. Sebebi, toplam doküman hacminin tek düğümde tutulamayacak kadar büyük olmasıdır. Yine de dokümanların kendi merkezi ortamınıza indirilmesi gerekir. Çünkü indeksleme için gereken metin ön işleme burada yapılacaktır.

10.2 Crawler'ın Temel Fikri

Sorguya karşılık sonuçları bulan arama bir indeks üzerinde yapılır. Kullanıcı sorguyu girip ilk 10 sonucu aldığında, bu sonuçlar indeksten gelir. Demek ki o kelimeye karşılık gelen dokü-

manları daha önceden indirmiş bir uygulama olmalıdır. Bu uygulama crawler'dır.

Crawler'ın çalışma mantığı şöyledir:

- Crawler'a başlangıç noktası olarak belli sayıda URL verilir. Bunlara tohum (seed) URL denir.
- Crawler bu URL'lere HTTP isteği (HTTP request) yapar. URL'ye karşılık gelen HTML dosyaları HTTP yanıt (HTTP response) nesnesinin içinde gelir.
- İndirilen sayfaların her biri ayrıştırılır (parse) ve içlerinden yeni URL'ler çıkarılır.
- Yeni URL'ler için de HTTP isteği yapılır, karşılık gelen dokümanlar indirilir.
- Bu dokümanlar da ayrıştırılır, yeni URL'ler çıkarılır ve döngü sürer.

Hoca slaytta bunu seviye seviye gösterdi. İndirme işlemi öyle yapılandırılmış olabilir ki, tohum URL'den başlayarak üç seviye gittikten sonra crawler durur. Bu, bir durma ölçütüdür (stopping criteria). O noktada indirilmiş doküman kümesi, normalde arama yapınca sonuçlarda göreceğiniz dokümanları içermelidir. Kullanıcının bir önceki slaytta yaptığı aramaya karşılık gelen doküman da crawler tarafından koleksiyon oluşturulurken bu aşamada indirilmiştir.

Özetle crawler, belli tohum URL'lerden başlayarak bir durma ölçütüne ulaşana kadar web'i tarayan, bir bağlantıdan diğerine giderek web'deki dokümanları sürekli merkezi bir yere indiren uygulamadır.

11. Gerçek Crawler'lar, Bant Genişliği ve robots.txt

11.1 Arama Motorlarının Crawler'ları

Crawler'lar her gün çalışan uygulamalardır. Her arama motorunun bir crawler'ı vardır. Hoca şu örnekleri verdi: Google için Googlebot, Bing için MSNbot, Yandex'in kendi crawler'ı. Altyazıda "Scooter" ve "Slug" diye iki ad daha geçiyor. İsimler genelde "bot" ile biter.

> **Ders sonrası not:** Hoca, Bing'in crawler'ının MSNbot olduğunu söyledi. Bu ad büyük olasılıkla eskidir; Bing'in bugünkü crawler'ı da büyük olasılıkla Bingbot adıyla bilinir. Hocanın arama motorunu adlandırmadığı "Scooter" büyük olasılıkla AltaVista'nın, altyazıda "Slug" diye geçen ad ise büyük olasılıkla Yahoo'nun "Slurp" adlı crawler'ıdır.

11.2 Bant Genişliği Kullanımı

Hoca, crawler uygulamalarının toplamda dünya genelindeki internet bant genişliğini en yoğun kullanan uygulamalar olduğunu söyledi. Gerekçesi şuydu: Crawler'lar sürekli çalışır. Crawler bir kez başlatılınca, bir sonraki indeks oluşturulana kadar sürekli HTTP isteği gönderir. Bunu tek bir crawler değil, çok sayıda crawler yapar. İstekler de dünya geneline dağılmış uygulama sunucularına gider. Bir web sayfasını yayınladığınızda (publish), onu bir uygulama sunucusu altında yayınlamış olursunuz. Haber sayfalarının da böyle sunucuları vardır. Crawler'lar bu sunucuları tarar.

> **Ders sonrası not:** Hoca, crawler'ların dünya genelindeki bant genişliğinde baskın (dominant) olduğunu söyledi. Altyazıda bunun için bir ölçüm verilmiyor. Doğrusu büyük olasılıkla şudur: Crawler'lar web isteklerinde önemli bir pay tutsa da, toplam internet trafiğinde bayt olarak baskın olan başlıca uygulama video akışıdır.

11.3 robots.txt

Crawler bir uygulama sunucusuna geldiğinde, sunucu altında hangi sayfaları tarayıp hangilerini tarayamayacağını gösteren bir doküman vardır. Bu `robots.txt` dosyasıdır. Crawler bu dosyaya bakarak yasak sayfaları görür ve yalnız tarayabileceği sayfaları tarar. Hoca önce "indeksleme" dedi, sonra ifadesini "crawl edebileceği sayfaları crawl ediyor" diye netleştirdi.

Aynı dosyada crawler'ın sayfayı hangi sıklıkta ziyaret edebileceğine dair bilgi de vardır. Hocanın sayfanın ağzından kurduğu örnek şöyleydi: "Arkadaş, arama sonuçlarında yukarıda çıkmam çok önemli değil, beni günde bir kez ziyaret et." Sayfa böyle diyorsa crawler buna uymak durumundadır. Aksi halde aynı sayfayı gereğinden fazla tarar ve uygulama sunucusunu boş

yere meşgul eder. Bu durumda sayfanın yöneticisi, o crawler'ın sayfayı taramasını yasaklayabilir.

> **Hocanın vurgusu:** Crawler'lar kullanıcı dostu (user friendly) olmalıdır. Bunun için her uygulama sunucusundaki `robots.txt` dosyasına bakıp hangi sayfaları ne sıklıkla tarayabileceklerine göre hareket ederler.
>
> **Ders sonrası not:** Hoca, dosyanın uygulama sunucusunun altında olduğunu ve crawler'ın buna uymak zorunda olduğunu söyledi. Doğrusu büyük olasılıkla şudur: `robots.txt` sitenin kök dizininde bulunan bir kurallar dosyasıdır ve kurallara uymak crawler'ın iyi davranışına bağlıdır, teknik olarak zorlanmaz. Ziyaret sıklığı için kullanılan `Crawl-delay` yönergesi de standart dışıdır ve her crawler tarafından dikkate alınmaz.

12. Crawler'ların Kullanım Alanları

Hoca motivasyonu kullanım alanlarıyla genişletti.

12.1 Genel Arama Motorları

Crawler, Google, Yahoo, Yandex ve Bing gibi evrensel (universal) arama motorlarını desteklemek için çalışır. Bu, en bilinen kullanımdır.

12.2 Dikey Arama Motorları

Yemek tarifleri, alışveriş siteleri ya da haberler üzerinde arama yapan bir motor geliştirmek istiyorsanız yine crawler kullanırsınız. Hocanın bir başka örneği Google Scholar'dı: Yalnız makaleler ve akademik yayınlar üzerinde arama yapar. Demek ki crawler'lar yalnız genel arama motorları için değil, dikey ya da özelleştirilmiş arama motorları için de doküman koleksiyonu oluşturur.

12.3 İş Zekası

Crawler bir iş zekası (business intelligence) aracı olarak da kullanılabilir. Hocanın örneği, fiyat politikasını belirlemek isteyen bir elektronik perakendecisiydi. Piyasadaki MediaMarkt, Teknosa, Vatan gibi rakiplerin fiyatlarından çok yüksek ya da çok düşük fiyat vermek istemezsiniz. Yapılacak iş şudur:

- Rakip firmaların sayfalarını ürün ve fiyat bazında sürekli tarayan bir crawler yazarsınız.
- Ürün bazında fiyat bilgilerini toplarsınız. Bunların örneğin ortalamasını ve standart sapmasını alırsınız.
- Kendi fiyatınızla karşılaştırırsınız. Fiyatınız onlardan çok yukarıda ya da çok aşağıdaysa düzeltirsiniz.
- Rakipler sizin satmadığınız ürünleri satıyorsa bunu fark eder, ürün çeşitliliğinizi artırırsınız.

Aynı şey online marketler için de geçerlidir. Bir market zinciri, Migros ve Carrefour gibi online satış da yapan süpermarketlerin neler sattığını izleyebilir, ürün çeşitliliği ve fiyat politikaları hakkında bilgi sahibi olabilir.

12.4 Web Sitelerindeki Değişiklikleri İzleme

Crawler'lar farklı web sitelerinde neler olup bittiğini izlemek için de kullanılır: Neler değişmiş, neler eklenmiş, haber sayfalarında bir önceki güne göre hangi yeni haberler gelmiş.

12.5 Pazarlama

Hocaya göre pazarlama dünyası crawler'ları çok kullanır. Satış yapmakla sorumlu olanlar, hedefledikleri kişilerin e-posta adreslerine ulaşmak ister. Bunun için sürekli e-posta adresi tarayıp bir e-posta veritabanı oluşturmak isterler. Bu veritabanıyla kişilere odaklı gönderim yapabilirler.

Hoca bunu kendi deneyiminden örnekledi: Akademik konferanslardan ve akademik yayın tercümesi yapan firmalardan okul hesaplarına sürekli e-postalar gelmektedir. Bu adreslerin okul tarafından bu firmalara verilmesi imkansızdır. Haberdar olmalarının başka yolu, sayfaları taramaktır. Bu firmalar Türkiye'deki üniversitelerin sayfaları için crawler yazar, akademisyenlerin e-posta adreslerine ulaşır ve tanıtım için toplu e-posta gönderir.

13. Crawler ve Arama Motoru Mimarisi

Hoca sonra bir crawler'ın bir arama motorunun içinde nerede durduğunu Google örneğiyle gösterdi. Önce, web'deki sayfalara HTTP isteği yapabilen ve belli bir crawler algoritmasına göre çalışan bir crawler olmalıdır.

- Crawler web'den veriyi alır ve bir sayfa deposuna (page repository) atar. Bu depo, bilgi erişimi mimarisindeki doküman koleksiyonudur.
- Depo üzerinde metin ön işleme yapılır. Sonucunda metin indeksi (text index), yani ters indeks oluşur.
- Depo üzerinde link analizi de yapılır. Sonucunda ikinci bir tablo oluşur: PageRank tablosu. Bu tablo HITS ya da başka bir bağlantı tabanlı algoritmayla da oluşturulabilir. Örnek Google olduğu ve Google PageRank tabanlı link analizi kullandığı için burada PageRank tablosu vardır.

```
web -> crawler -> sayfa deposu -> metin ön işleme -> ters indeks
                        |
                        +-------> link analizi ----> PageRank tablosu
```

Arama anında akış şöyledir. Kullanıcı sorguyu girer. Arama motoru ters indeks üzerinde lookup yapar ve sorguya karşılık gelen sayfaları tespit eder. Bu sonuçlar doğrudan kullanıcıya gösterilmez. PageRank sonuçlarıyla birleştirilir ve yeniden sıralanır. Yani iki iş yapılır: Önce sorguya karşılık gelen sayfalar bulunur, sonra bu sayfalar hem TF-IDF hem PageRank skorları dikkate alınarak sıralanır. Kullanıcının gördüğü liste bu sıralanmış sonuçlardır.

14. Crawler Sınıflandırması

Farklı motivasyonları karşılamak için literatürde farklı crawler'lar önerilmiş, endüstride de gerçekleştirilmiştir. Hoca sınıflandırmayı şöyle kurdu:

```
crawler
├── universal crawler
└── preferential crawler
    ├── focused crawler
    └── topical crawler
        ├── adaptif
        └── statik
```

14.1 Universal Crawler

Yahoo, Yandex ve Google gibi genel arama motorlarını destekler. Arka planda bu motorların doküman koleksiyonlarını oluşturur. Hiçbir ayrım yapmadan bütün web sitelerinden indeks oluşturur.

14.2 Preferential Crawler

Dikey arama motorlarının doküman koleksiyonlarını oluşturur. Kendi içinde focused ve topical diye ikiye ayrılır.

14.3 Focused Crawler

Yemek tariflerinde, elektronik ürünlerde ya da süpermarket ürünlerinde arama yapan motorların doküman koleksiyonlarını oluşturur. Arama motoru hangi konu (topic) için geliştiriliyorsa, o konunun dokümanlarını indirir ve indeks oluşturur.

> **Hocanın vurgusu:** Hem universal hem focused crawler'da indeks önceden oluşturulur ve arama bu indeks üzerinde gerçekleşir. İkisi arasındaki fark, universal crawler'ın ayrım yapmaması, focused crawler'ın ise yalnız seçilen konuya bakmasıdır.

14.4 Topical Crawler

Topical crawler'da indeks önceden oluşturulmaz, sorgu anında oluşturulur. Sorgu girildiğinde web taranmaya başlanır. Sorguya uyan sonuçlar adım adım gösterilir.

Hoca derste bir örnek gösterdi. Bu araçta, aradığınız anahtar kelimeye karşılık web'de yayınlananlar sizin adınıza sürekli izlenir. Örneğin "the north face" sorgusu girdiğinizde, ifadenin hangi sayfalarda geçtiği gerçek zamanlı olarak aranır ve gösterilir. Araç bir indeks oluşturmaz. Sorguyu ne zaman girdiyseniz web'i o zaman taramaya başlar.

İki yöntem arasındaki fark hocanın sözleriyle şöyleydi. İndeksli yöntemde bir gün öncesinden bütün dokümanlar indirilir, bir indeks oluşturulur ve arama bu indeks üzerinde yapılır. Topical yöntemde ise web o anda aranır. Karşınıza çıkan, sorguya uyan dokümanlar sıralanarak listelenir.

Topical crawler'ın alt kolları vardır:

- **Adaptif (adaptive):** Arama yaparken, taranan sayfalardan öğrenilen yeni bilgiler de dikkate alınarak aramaya devam edilir.
- **Statik (static):** Genişlik öncelikli arama (breadth-first search), derinlik öncelikli arama (depth-first search) ya da link analizi skorlarına bakılarak sayfadan sayfaya gidilir.

Hoca crawler'ın taksonomi sayfasında durdu. Bu konular sonraki derslerde daha kapsamlı işlenecektir.

15. Crawler ve Scraper Sorusu

Soru: Crawler ile scraper arasındaki fark nedir? Sorunun başı altyazıda eksiktir, ama devamından anlaşılan soru budur.

Cevap: Hoca scraper terimini bilmediğini söyledi. Soruyu soran tanımlayabilirse ya da kendisi baktıktan sonra bu konuyu ayrıca konuşabileceklerini belirtti.

> **Ek bilgi (derste anlatılmadı):** Yaygın kullanımda crawler, bağlantıları izleyerek sayfaları keşfeden ve indiren uygulamadır. Scraper ise belli sayfalardan belirli alanları, örneğin ürün adı ve fiyatı, yapılandırılmış biçimde çıkaran uygulamadır. İkisi birlikte de kullanılabilir: Crawler sayfaları bulur, scraper bu sayfalardan veriyi alır. 12.3'teki fiyat izleme örneği bu ikisinin birleşimine benzer.

16. Proje Önerisi Görüşmesi

Dersin sonunda hoca, projesini konuşmak isteyenlere söz verdi. Bir öğrenci ekranını paylaşarak önerisini gösterdi.

16.1 1. Sunum

- **Konu:** Yayınlanan uygulamaların (application) kullanıcılardan alacağı puanın tahmini.
- **Amaç:** Kullanıcılar yayınlanan uygulamaları puanlıyor ve her uygulama belli bir puana sahip oluyor. Öğrenci, bir uygulamanın son olarak alacağı puanı önceden kestirmek istiyor. Puan 1 ile 5 arasında sayısal bir değerdir.
- **Yöntem:** Hazır bir veri setiyle çalışılacak ve Python ile yazılacak. Kütüphane olarak Pandas kullanılacak. Öğrenci, sınıflandırma algoritmalarının yanında doğrusal regresyonu (linear regression) da düşündüğünü ve alandaki önceki projelerden yararlanacağını söyledi.
- **Sonuç:** Proje henüz öneri aşamasındadır. Elde edilmiş bir sonuç sunulmadı.

16.2 Hocanın Geri Bildirimleri

Hoca, önce problemin türünü netleştirdi. Sayısal değer tahmini yapılacaksa regresyon analizi ya da LSTM tabanlı sayısal değer tahmini kullanılmalıdır. Öneride gösterilen algoritmalar ise sınıflandırma (classification) algoritmalarıdır. Bunlar kategorik değer tahmin eder: İkili sınıflandırmada evet ya da hayır, kabul ya da red, mavi ya da kırmızı gibi.

Problem başka bir biçimde de kurulabilir. Puan 1, 2, 3, 4 ya da 5 değerlerinden biri olacaksa ve arada 1,1 ya da 1,5 gibi gerçel sayılar yoksa, her puan ayrı bir kategori sayılır. Bu durumda problem çok sınıflı (multi-class) tahmine döner ve bu tür algoritmalar kullanılabilir. Ama ikili tahmin yapan yöntemin çok sınıflı tahmin yapacak biçime dönüştürülmesi gerekir.

Soru: Puanları kategorize edip kategorilerden birini seçmek mi, yoksa doğrudan sayısal değeri bulmak mı tercih edilmeli?

Cevap: Hocanın bir tercihi yok. Regresyon analizi de kategorik tahmin yapan algoritmalar da kabul edilir. Ama seçilen yaklaşımda en az 3 algoritma kullanılmalıdır.

> **Hocanın vurgusu:** "Kategori tahmin eden sınıflandırma algoritmalarını kullanarak sayısal değer tahmini yapıyorum" denmemelidir. Elma ile armut karıştırılmamalıdır.

Hoca kabul edeceği anlatımı da tarif etti. Öğrenci random forest, k en yakın komşu ve destek vektör makinesi (support vector machine) kullanacağını, bunların kategorik değer tahmin ettiğini, her puanı bir kategori olarak değerlendirip problemi çok sınıflı problem olarak ele aldığını söylemelidir. Ardından, çok sınıflı tahmin yapan makaleleri göstermelidir: Bu algoritmaların beş etiketten birini tahmin etmek için nasıl kullanıldığını, özellik kümesinin (feature set) ne olduğunu ve gerçekleştirmenin bu makalelerden yola çıkarak nasıl yapıldığını anlatmalıdır. Yani seçilen yol literatüre dayandırılmalıdır.

> **Ders sonrası not:** Hoca çok sınıflı durum için yer yer "multi label" dedi. Doğrusu "multi-class" olmalıdır. Burada her örneğe beş etiketten yalnız biri verilmektedir. Çok etiketli (multi-label) sınıflandırmada ise bir örneğe aynı anda birden fazla etiket verilebilir.

> **Ders sonrası not:** Hoca random forest, k en yakın komşu ve destek vektör makinesini kategorik değer tahmin eden algoritmalar olarak anlattı. Bu, sınıflandırma kullanımları için doğrudur. Ancak büyük olasılıkla bu yöntemlerin regresyon sürümleri de vardır ve sayısal değer tahmini için de kullanılabilirler. Önemli olan, kullanılan sürümün problemin türüne uymasıdır.

Dersin Toparlaması

Bu derste bilgi erişimi bölümü tamamlandı ve web crawling bölümüne girildi. Bir doküman, terimlerin çıkarılması, durak kelimelerin atılması, kök bulma ve TF-IDF hesabıyla temsilci vektöre dönüştürülür. Vektörlerin oluşturduğu doküman-terim matrisinden ters indeks kurulur. Arama bu indeks üzerinde lookup, birleştirme ve yeniden sıralama adımlarıyla yapılır. Motorun başarısı precision ve recall ile ölçülür. Web ortamında recall ölçülemediği için Precision@10 öne çıkar, çeşitlilik de karşılaştırmaya girer. Eğri ise çok sayıda sorgunun ortalamasından çizilir.

İkinci yarıda doküman koleksiyonunun nasıl oluştuğu konuşuldu. Crawler tohum URL'lerden başlar, sayfaları indirip içlerinden yeni URL'ler çıkarır ve bir durma ölçütüne kadar devam eder. Bunu yaparken `robots.txt` kurallarına uyar. Crawler'lar universal ve preferential diye ayrılır, preferential olanlar da focused ve topical diye ikiye ayrılır. Sonraki işlenecek derste crawler'lar daha kapsamlı ele alınacaktır. Link analizi ve PageRank tablosunun ayrıntıları ise web yapı madenciliği konuşulurken görülecektir.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.